Full-Stack Blog System

A modern, feature-rich blog platform built with Rust (Axum) backend and Next.js frontend, powered by Turso (LibSQL) edge database.

Features

Frontend (Next.js 15 + React 19)

- **Modern UI** - Clean, responsive design with Tailwind CSS
- **Markdown Editor** - Live preview with split-view editing
- **Image Management** - Drag & drop uploads with automatic optimization
- **SEO Optimized** - Meta tags, OpenGraph, sitemap, RSS feed
- **Dark/Light Mode** - User preference support
- **Fast Loading** - Optimized images (WebP), lazy loading
- **Mobile First** - Fully responsive design

Admin Panel

- **Authentication** - Secure JWT-based auth with bcrypt
- **Post Management** - Create, edit, delete with live markdown preview
- **Draft System** - Save drafts, publish when ready
- **Image Gallery** - Upload, search, batch operations
- **Analytics Dashboard** - Views, storage stats, recent uploads
- **Comment Moderation** - Approve/delete comments
- **User Management** - Role-based access control

Image System

- **Smart Upload** - Drag & drop, multiple files (up to 20)
- **Auto Optimization** - 5 variants per image:
  - Original
  - Thumbnail (150px)
  - Small (400px)
  - Medium (800px)
  - WebP format (30-80% smaller)
- **Batch Operations** - Select multiple, copy URLs, delete
- **Search & Filter** - By filename, date range, size
- **Analytics** - Storage usage, largest images, orphaned files

Backend (Rust + Axum)

- **Fast & Efficient** - Rust's performance and safety
- **RESTful API** - Clean, documented endpoints
- **Database** - Turso (LibSQL) edge database with replication
- **Image Storage** - Base64 in database with caching

- **RSS Feed** - Auto-generated from posts
- **CORS Support** - Configurable for frontend

Performance

- **Parallel Uploads** - 4-5x faster than sequential
- **WebP Conversion** - 30-80% file size reduction
- **Image Caching** - 1-year browser cache
- **Client-side Filtering** - Sub-10ms search/filter
- **Edge Database** - Global low-latency access

Table of Contents

Quick Start

Prerequisites

- **Rust** 1.75+ (Install)
- **Node.js** 18+ (Install)
- **Turso CLI** (Install)

1. Clone & Setup

```
# Clone repository
git clone https://github.com/azurewood/blog-system.git
cd blog-system

# Setup Turso database
turso auth login
turso db create blog-db
turso db show blog-db --url      # Save this URL
turso db tokens create blog-db   # Save this token
```

2. Configure Backend

```
cd backend

# Copy environment template
cp .env.example .env

# Edit .env with your values:
# TURSO_DATABASE_URL=libsql://your-db.turso.io
# TURSO_AUTH_TOKEN=your-token-here
# JWT_SECRET=your-secret-key
# PORT=3001
```

3. Initialize Database

```
# Initialize schema and create admin user
cargo run --bin setup

# Enter admin credentials when prompted:
# Email: admin@example.com
# Password: your-secure-password
```

4. Start Backend

```
cargo run

# Server running at http://localhost:3001
```

5. Configure Frontend

```
cd frontend

# Copy environment template
cp .env.local.example .env.local

# Edit .env.local:
# NEXT_PUBLIC_API_URL=http://localhost:3001
```

6. Start Frontend

```
npm install
npm run dev

# App running at http://localhost:3000
```

7. Access Admin Panel

```
URL: http://localhost:3000/login
```

```
Email: admin@example.com
Password: your-secure-password
```

Installation

Backend Setup

```
cd backend

# Install dependencies (handled by Cargo)
cargo build

# Run database setup
cargo run --bin setup

# Start development server
cargo run

# Or build for production
cargo build --release
./target/release/blog-backend
```

Frontend Setup

```
cd frontend

# Install dependencies
npm install

# Development mode
npm run dev

# Production build
npm run build
npm start
```

Configuration

Environment Variables

Backend `.env`

```
# Database
TURSO_DATABASE_URL=libsql://your-database.turso.io
TURSO_AUTH_TOKEN=your-turso-token
```

```
# Server
PORT=3001
API_BASE_URL=http://localhost:3001

# Security
JWT_SECRET=your-very-secret-key-min-32-chars

# Optional
RUST_LOG=info
```

Frontend `.env.local`

```
# API Connection
NEXT_PUBLIC_API_URL=http://localhost:3001

# Optional: Analytics
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX
```

Database Schema

The database uses the following tables: - `users` - User accounts and authentication - `posts` - Blog posts with metadata - `comments` - Post comments (CASCADE delete) - `tags` - Post tags - `post_tags` - Many-to-many relationship - `images` - Image storage with variants

Cascade Deletion: - Deleting a post automatically deletes its comments and tag relationships - Foreign keys configured with `ON DELETE CASCADE`

Development

Backend Development

```
cd backend

# Watch mode (auto-restart on changes)
cargo watch -x run

# Run tests
cargo test

# Format code
cargo fmt

# Lint
cargo clippy
```

```
# Check without building
cargo check
```

Frontend Development

```
cd frontend

# Development server with hot reload
npm run dev

# Type checking
npm run type-check

# Linting
npm run lint

# Build for production
npm run build
```

Database Management

```
# Open Turso shell
turso db shell blog-db

# Common queries
SELECT COUNT(*) FROM posts;
SELECT * FROM users;
SELECT * FROM comments WHERE is_approved = 0;

# Backup database
turso db shell blog-db ".dump" > backup.sql

# List all databases
turso db list

# Show database info
turso db show blog-db
```

Deployment

Recommended: Fly.io + Vercel

Architecture:

```
Frontend (Vercel) → Backend (Fly.io) → Database (Turso)
```

Deploy Backend to Fly.io

```
cd backend

# Install Fly CLI
brew install flyctl  # macOS
# or: curl -L https://fly.io/install.sh | sh

# Login
fly auth login

# Create app
fly launch

# Set secrets
fly secrets set \
  TURSO_DATABASE_URL="libsql://your-db.turso.io" \
  TURSO_AUTH_TOKEN="your-token" \
  JWT_SECRET="your-secret"

# Deploy
fly deploy

# Your backend: https://your-app.fly.dev
```

Deploy Frontend to Vercel

```
cd frontend

# Install Vercel CLI
npm install -g vercel

# Deploy
vercel

# Add environment variable in Vercel dashboard:
# NEXT_PUBLIC_API_URL=https://your-backend.fly.dev

# Production deploy
vercel --prod

# Your blog: https://your-blog.vercel.app
```

Automated Deployment Script

```
# Use the included deployment script
./deploy.sh
```

```
# Follows prompts to:
# 1. Deploy backend to Fly.io
# 2. Deploy frontend to Vercel
# 3. Set up environment variables
```

Alternative: Railway

```
# Install Railway CLI
npm install -g @railway/cli

# Login
railway login

# Deploy backend
cd backend
railway init
railway up

# Deploy frontend
cd frontend
railway init
railway up
```

Features Guide

Creating Posts

1. **Login** to admin panel at `/login`
2. **Navigate** to Posts → New Post
3. **Write** content using Markdown
4. **Preview** in real-time with split-view editor
5. **Upload** featured image
6. **Add** tags (comma-separated)
7. **Save** as draft or publish immediately

Markdown Editor

The admin panel includes a powerful markdown editor with:

- **Split View** - Edit and preview side-by-side
- **Live Preview** - See rendered output in real-time
- **Syntax Support**:
  - Headers (H1-H6)
  - Bold, italic, strikethrough
  - Lists (ordered, unordered, task lists)

- Code blocks with syntax highlighting
- Links and images
- Tables
- Blockquotes
- HTML (sanitized)

View Modes: - Edit Only - Just the markdown editor - Split View - Editor + Preview (default) - Preview Only - Rendered output only

Quick Reference:

```
# Headers
## Level 2
### Level 3

**Bold** and *italic*

- Bullet list
1. Numbered list
- [ ] Task list

[Link](https://example.com)
![Image](url)

`inline code`

```javascript
// Code block
console.log("Hello");
```

> Blockquote
```

```
### Image Management

**Upload Images:**
1. Go to Admin → Images
2. Drag & drop files or click to select
3. Upload up to 20 images at once
4. Automatic optimization creates 5 variants

**Search & Filter:**
- Filter by filename
- Filter by date range
- Filter by file size
- Filter by variant type

**Batch Operations:**
- Select multiple images
```

```
- Copy URLs to clipboard
- Delete multiple at once

**Analytics:**
- Total storage used
- Number of images
- Largest files
- Recent uploads
- Orphaned images (not used in posts)

### Comment Moderation

**Enable comments** on blog posts, then:

1. Go to Admin → Comments
2. View pending comments
3. Approve or delete
4. Approved comments appear on posts

### Deleting Posts

**From Posts List:**
1. Click "Delete" next to any post
2. Confirm deletion in modal
3. Post and all comments are deleted (CASCADE)

**What Gets Deleted:**
- The post itself
- All comments on the post
- Post-tag relationships
- (Featured image remains in gallery)

---

##   API Documentation

### Authentication

**Login**
```http
POST /api/auth/login
Content-Type: application/json

{
  "email": "admin@example.com",
  "password": "password123"
```
```

```
}

Response:
{
  "success": true,
  "data": {
    "token": "eyJhbGci...",
    "user": {
      "id": "...",
      "email": "admin@example.com",
      "role": "admin"
    }
  }
}
```

Posts

List Posts

```
GET /api/posts?limit=10&offset=0

Response:
{
  "success": true,
  "data": [
    {
      "id": "...",
      "title": "Post Title",
      "slug": "post-title",
      "content": "...",
      "status": "published",
      "views": 42,
      "created_at": 1234567890
    }
  ]
}
```

Get Post by Slug

```
GET /api/posts/by-slug/{slug}
```

Get Post by ID

```
GET /api/posts/{id}
```

Create Post (Requires auth)

```
POST /api/posts
Authorization: Bearer {token}
```

```
Content-Type: application/json

{
  "title": "New Post",
  "slug": "new-post",
  "content": "# Hello World",
  "excerpt": "A short description",
  "status": "published",
  "tags": ["tech", "blog"]
}
```

Update Post (Requires auth)

```
PUT /api/posts/{id}
Authorization: Bearer {token}
Content-Type: application/json

{
  "title": "Updated Title",
  "content": "Updated content"
}
```

Delete Post (Requires auth)

```
DELETE /api/posts/{id}
Authorization: Bearer {token}

Response:
{
  "success": true,
  "data": "Post deleted successfully"
}
```

Images

Upload Image (Requires auth)

```
POST /api/images
Authorization: Bearer {token}
Content-Type: application/json

{
  "filename": "image.jpg",
  "data": "base64-encoded-data"
}

Response:
{
```

```
  "success": true,
  "data": {
    "id": "...",
    "original": "/api/images/{id}/original",
    "thumbnail": "/api/images/{id}/thumbnail",
    "small": "/api/images/{id}/small",
    "medium": "/api/images/{id}/medium",
    "webp": "/api/images/{id}/webp"
  }
}
```

Get Image

```
GET /api/images/{id}/{variant}

Variants: original, thumbnail, small, medium, webp
```

Image Analytics (Requires auth)

```
GET /api/images/analytics
Authorization: Bearer {token}

Response:
{
  "success": true,
  "data": {
    "total_size_bytes": 12345678,
    "total_images": 50,
    "by_variant": {...},
    "largest_images": [...],
    "recent_uploads": [...]
  }
}
```

Delete Image (Requires auth)

```
DELETE /api/images/{id}
Authorization: Bearer {token}
```

Comments

Get Post Comments

```
GET /api/posts/{post_id}/comments
```

Create Comment

```
POST /api/posts/{post_id}/comments
Content-Type: application/json
```

```
{
  "author_name": "John Doe",
  "author_email": "john@example.com",
  "content": "Great post!"
}
```

Approve Comment (Requires auth)

```
PUT /api/comments/{id}/approve
Authorization: Bearer {token}
```

Delete Comment (Requires auth)

```
DELETE /api/comments/{id}
Authorization: Bearer {token}
```

RSS Feed

```
GET /feed.xml

Returns RSS 2.0 XML feed of published posts
```

Troubleshooting

Common Issues

Database Connection Error **Error:** `The tls feature is disabled`

Fix:

```toml
# In backend/Cargo.toml, ensure:
libsql = { version = "0.6", features = ["core", "replication", "sync"] }
```

Then:

```
cd backend
cargo clean
cargo build
```

Database Corruption **Error:** `SQLITE_CORRUPT: database disk image is malformed`

Fix:

```
cd backend
./reset-database.sh

# Or manually:
rm -f local.db
cargo run --bin setup
```

Prevention: Use Turso Cloud instead of local SQLite

Authentication Fails **Error:** `Unauthenticated` when editing posts

Fix: - Clear browser localStorage - Logout and login again - Check token in browser DevTools console: `javascript localStorage.getItem('auth_token')`

JSON Parse Error on Delete **Error:** `SyntaxError: Unexpected end of JSON input`

Fix: Ensure foreign keys have CASCADE:

```
-- In Turso shell
.schema comments
-- Should show: ON DELETE CASCADE

-- If not, recreate table with CASCADE
-- See schema.sql
```

Port Already in Use **Error:** `Address already in use`

Fix:

```
# Find process using port
lsof -i :3001  # Backend
lsof -i :3000  # Frontend

# Kill process
kill -9 {PID}
```

Images Not Loading **Check:** 1. Image uploaded successfully (check browser Network tab) 2. Backend running and accessible 3. CORS configured correctly 4. Image variant exists

Debug:

```
# Test image endpoint
curl http://localhost:3001/api/images/{id}/thumbnail

# Check analytics
curl http://localhost:3001/api/images/analytics -H "Authorization: Bearer {token}"
```

Reset Everything

```
# Backend
cd backend
cargo clean
rm -f local.db*
cargo run --bin setup
```

```
# Frontend
cd frontend
rm -rf .next node_modules
npm install
npm run dev
```

Additional Documentation

- **VERCEL_DEPLOYMENT.md** - Complete deployment guide
- **DEPLOYMENT_CHECKLIST.md** - Step-by-step deployment checklist
- **DATABASE_CORRUPTION_FIX.md** - Database troubleshooting
- **LIBSQL_TLS_FIX.md** - TLS configuration guide
- **AUTH_FIX.md** - Authentication troubleshooting
- **QUICK_DATABASE_FIX.md** - Quick database reset guide
- **VERCEL_RUST_ALTERNATIVES.md** - Deployment alternatives

Contributing

Contributions welcome! Please:

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests if applicable
5. Submit a pull request

Code Style

Rust:

```
cargo fmt
cargo clippy
```

TypeScript:

```
npm run lint
npm run type-check
```

License

MIT License - see LICENSE file for details

Acknowledgments

- Built with Axum
- Frontend powered by Next.js
- Database by Turso
- UI components from Tailwind CSS
- Markdown rendering with react-markdown

Support

- **Issues:** GitHub Issues
- **Discussions:** GitHub Discussions
- **Email:** jian.zhou@lincolnuni.ac.nz

Roadmap

- ☐ Multi-user support
- ☐ Categories/taxonomies
- ☐ Email notifications
- ☐ Social media integration
- ☐ Advanced analytics
- ☐ Import/export functionality
- ☐ Multi-language support
- ☐ Custom themes
- ☐ Plugin system

Built with using Rust and Next.js

Star this repo if you find it useful!